

Rapport technique détaillé

Élément	Information
Date du rapport	21/05/2026
Période Git observée	2026-03-04 au 2026-06-26
Dépôt GitHub	https://github.com/HoSSamOuammi/Digital_Thinking
Objectif	Présenter l'application, ses modules, son pipeline technique et les choix faits pendant le projet.

1. Concept et direction artistique

Nous avons pensé l'application comme un petit studio numérique. L'utilisateur peut partir de réglages simples, d'un fichier CSV ou d'une image, produire un rendu, puis le retrouver dans une galerie. Le projet reste volontairement accessible: il doit se tester rapidement et se comprendre sans lire tout le code.

La direction visuelle est sobre: fond clair, cartes lisibles, navigation stable et textes français. Ce choix donne une impression d'outil terminé, sans transformer le projet en page promotionnelle trop chargée.

2. Vue d'ensemble des modules

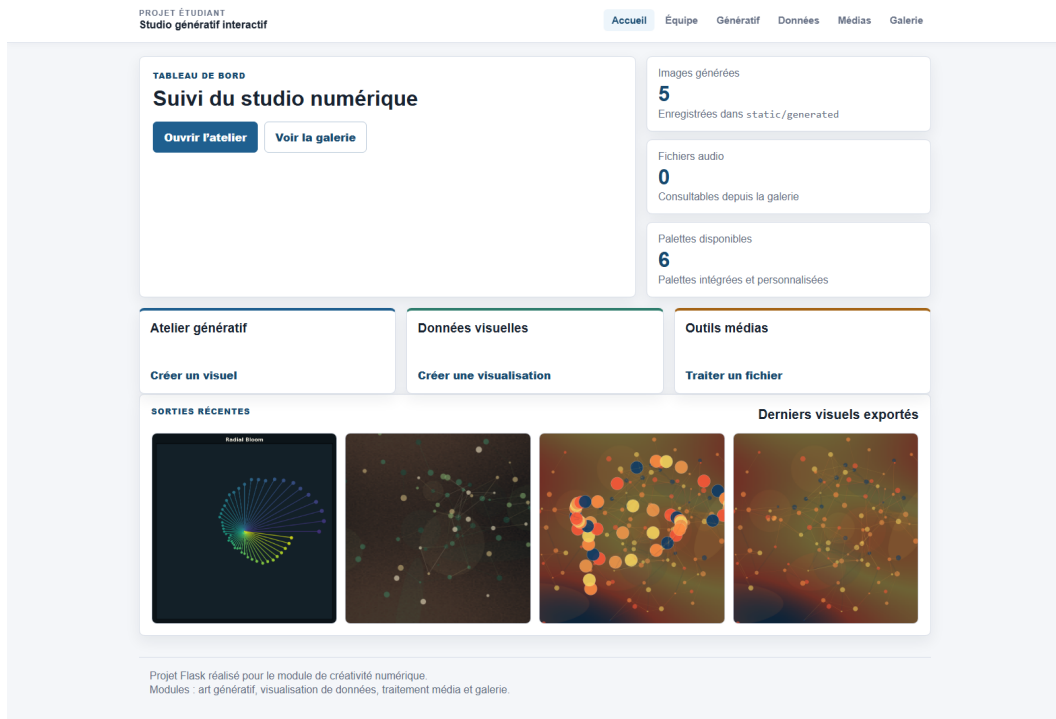
Module	Rôle dans le projet
Tableau de bord	La page d'accueil donne le ton du projet. On y retrouve les compteurs, les accès aux ateliers et les derniers exports. C'est volontairement simple: quelqu'un qui découvre le projet doit comprendre en quelques secondes où cliquer pour tester l'application.
Atelier génératif	L'atelier génératif est la partie la plus créative. L'utilisateur règle une série visuelle, une palette, un fond, une graine et plusieurs paramètres de densité ou de taille. La prévisualisation aide à tester rapidement une idée avant de lancer l'export final.
Données visuelles	Ce module transforme un CSV, ou un jeu de démonstration, en image. Les valeurs numériques sont nettoyées puis utilisées pour produire une visualisation. L'intérêt est de montrer que le studio ne génère pas seulement des formes abstraites: il peut aussi partir de données.
Outils médias	La page médias permet d'importer une image et d'appliquer des effets visibles: sépia, contours, rotation, glitch, palette dominante, etc. Le traitement audio reste prévu, mais il dépend de ffmpeg, donc l'application affiche clairement l'état de disponibilité au lieu de planter.
Galerie	La galerie ferme le parcours utilisateur. Après une génération ou un traitement, les fichiers sont listés avec pagination et liens de téléchargement. Cette page prouve que le flux complet fonctionne: créer, sauvegarder, retrouver, télécharger.
Équipe	La page équipe présente les membres, leurs rôles, leurs emails et leurs photos. Elle rend le projet plus humain et permet de relier les parties techniques à la répartition réelle du travail.

3. Parcours illustré de l'application

3.1 Tableau de bord

La page d'accueil donne le ton du projet. On y retrouve les compteurs, les accès aux ateliers et les derniers exports. C'est volontairement simple: quelqu'un qui découvre le projet doit comprendre en quelques secondes où cliquer pour tester l'application.

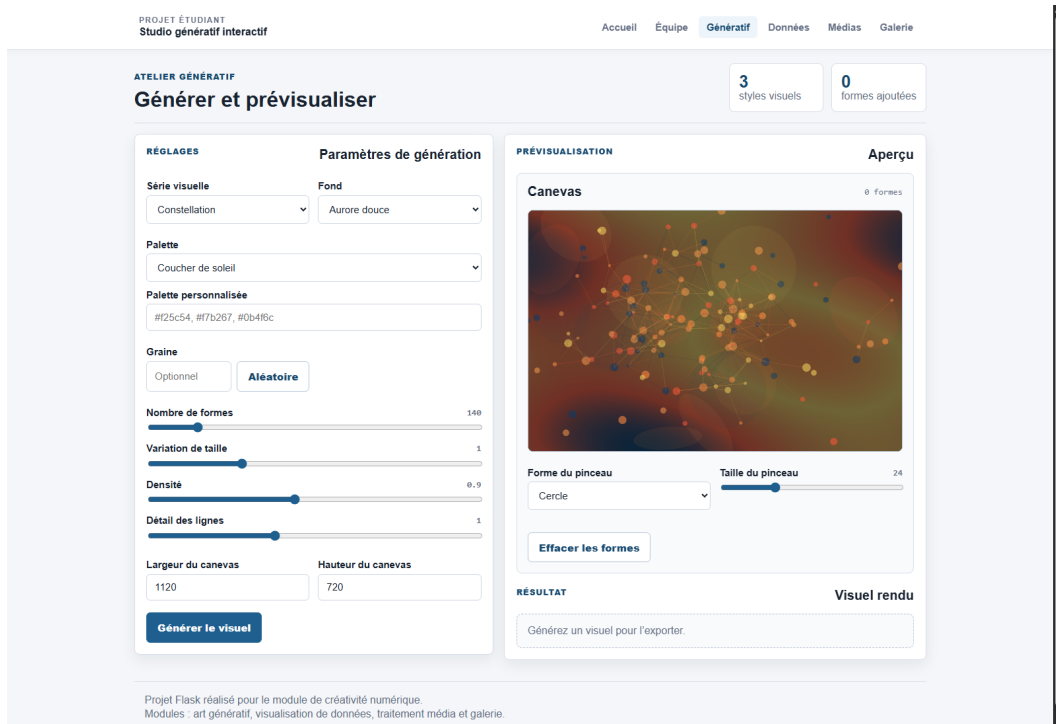
Capture de l'application - Tableau de bord



3.2 Atelier génératif

L'atelier génératif est la partie la plus créative. L'utilisateur règle une série visuelle, une palette, un fond, une graine et plusieurs paramètres de densité ou de taille. La prévisualisation aide à tester rapidement une idée avant de lancer l'export final.

Capture de l'application - Atelier génératif



3.3 Données visuelles

Ce module transforme un CSV, ou un jeu de démonstration, en image. Les valeurs numériques sont nettoyées puis utilisées pour produire une visualisation. L'intérêt est de montrer que le studio ne génère pas seulement des formes abstraites: il peut aussi partir de données.

Capture de l'application - Données visuelles

The screenshot shows the 'Données visuelles' module interface. At the top, there's a navigation bar with 'Accueil', 'Équipe', 'Génératif', 'Données' (active), 'Médias', and 'Galerie'. Below the navigation bar, the title 'DONNÉES VISUELLES' is followed by 'Transformer les données'. On the right, there are two counters: '5 colonnes numériques' and '5 palettes couleur'. The main content area is divided into two panels. The left panel, titled 'RÉGLAGES' and 'Préparation des données', contains settings for 'Style de visualisation' (set to 'Vue complète'), 'Colonne principale' (set to 'Automatique'), 'Palette Matplotlib' (set to 'magma'), and 'Fenêtre de lissage' (set to '8'). There is also a section for 'Jeu de données CSV (optionnel)' with a 'Choose File' button and a 'No file chosen' message. A 'Générer la visualisation' button is at the bottom. The right panel, titled 'RÉSULTAT' and 'Visualisation rendue', contains a text box with the instruction 'Générez une visualisation pour l'afficher ici.' and a message 'Colonnes détectées : temperature, humidity, wind, traffic, energy'. At the bottom, there is a footer with the text 'Projet Flask réalisé pour le module de créativité numérique. Modules : art génératif, visualisation de données, traitement média et galerie.'

3.4 Outils médias

La page médias permet d'importer une image et d'appliquer des effets visibles: sépia, contours, rotation, glitch, palette dominante, etc. Le traitement audio reste prévu, mais il dépend de ffmpeg, donc l'application affiche clairement l'état de disponibilité au lieu de planter.

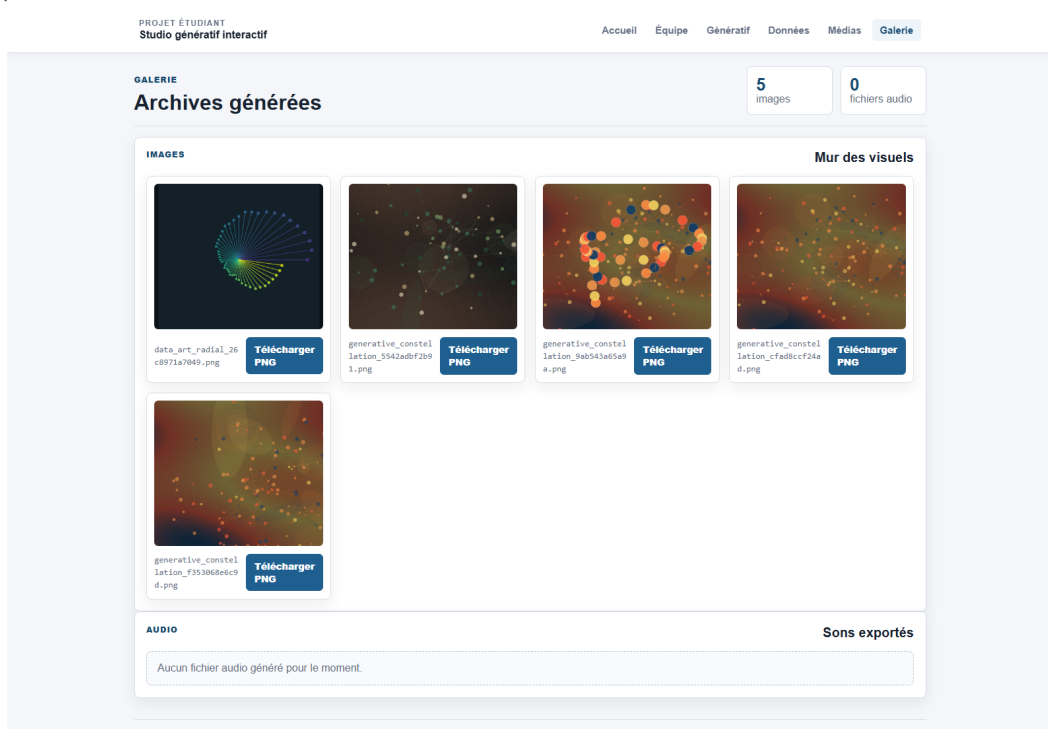
Capture de l'application - Outils médias

The screenshot shows the 'Outils médias' module interface. At the top, there's a navigation bar with 'Accueil', 'Équipe', 'Génératif', 'Données', 'Médias' (active), and 'Galerie'. Below the navigation bar, the title 'OUTILS MÉDIAS' is followed by 'Traiter les fichiers'. On the right, there are two counters: '13 effets image' and '0 opérations audio'. The main content area is divided into two panels. The left panel, titled 'IMAGE' and 'Importer et transformer', contains a 'Fichier image' section with a 'Choose File' button and a 'No file chosen' message. Below that, there is an 'Effet' dropdown menu set to 'Néon'. An 'Appliquer l'effet image' button is at the bottom. The right panel, titled 'AUDIO' and 'Traitement sonore', contains a message 'Les outils audio sont désactivés. PyDub n'est pas installé.' At the bottom, there is a footer with the text 'Projet Flask réalisé pour le module de créativité numérique. Modules : art génératif, visualisation de données, traitement média et galerie.'

3.5 Galerie

La galerie ferme le parcours utilisateur. Après une génération ou un traitement, les fichiers sont listés avec pagination et liens de téléchargement. Cette page prouve que le flux complet fonctionne: créer, sauvegarder, retrouver, télécharger.

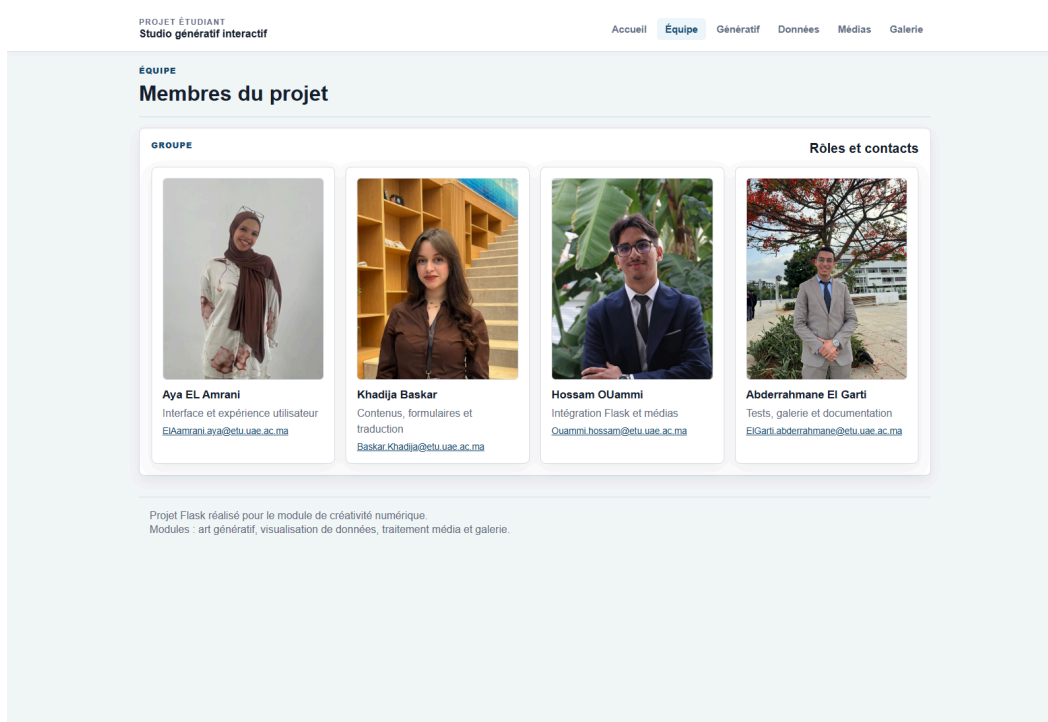
Capture de l'application - Galerie



3.6 Équipe

La page équipe présente les membres, leurs rôles, leurs emails et leurs photos. Elle rend le projet plus humain et permet de relier les parties techniques à la répartition réelle du travail.

Capture de l'application - Équipe



4. Architecture et pipeline technique

Le projet a été séparé pour éviter un seul fichier trop long. `app.py` lance l'application, `studio/app\_factory.py` prépare Flask, `studio/routes/` contient les vues, et `modules/` garde les traitements métier. Cette structure reste simple, mais elle donne une vraie logique au projet.

Fichier ou dossier	Rôle
app.py	Point d'entrée de l'application.
studio/app_factory.py	Création de Flask et enregistrement des routes.
studio/routes/	Pages séparées par fonctionnalité.
studio/forms.py	Lecture et validation des paramètres.
studio/storage.py	Sauvegarde, pagination et nettoyage des fichiers.
studio/security.py	Protection CSRF des formulaires.
modules/	Génération artistique, données, image et audio.
templates/	Pages HTML Jinja2.
static/	CSS, photos, captures, exports et fichiers générés.

Pipeline de fonctionnement

- L'utilisateur choisit un atelier depuis le tableau de bord.
- Le formulaire est envoyé à Flask avec un jeton CSRF.
- Les paramètres sont lus et normalisés dans studio/forms.py.
- Le module métier correspondant génère ou transforme le contenu.
- Le fichier obtenu est sauvegardé dans static/generated.
- La page affiche le résultat et propose le téléchargement.
- Les tests rejouent les parcours importants pour éviter les régressions.

5. Outils utilisés

Outil	Utilisation
Flask	Routage, formulaires, sessions et rendu des pages.
Jinja2	Templates HTML avec données envoyées par Flask.
Pillow	Lecture, transformation et export d'images.
Pandas / NumPy	Préparation des données CSV et calculs numériques.
Matplotlib	Création des visualisations exportées en image.
PyDub / ffmpeg	Traitement audio lorsque l'environnement le permet.
unittest	Tests fonctionnels des pages et traitements principaux.
Git / GitHub	Historique de travail, dépôt final et preuves de collaboration.

6. Challenges et solutions

Les difficultés principales n'étaient pas seulement techniques. Il fallait aussi rendre le projet lisible, présentable et assez stable pour être lancé pendant une soutenance.

Challenge	Solution
Le fichier app.py était trop chargé	La logique a été déplacée vers une fabrique Flask, des fichiers de routes et des services simples. Le projet est plus facile à lire et à expliquer.
L'interface devait paraître terminée	Les textes ont été harmonisés en français et le design a été rendu plus sobre pour ressembler à un vrai outil étudiant.
Les exports pouvaient encombrer le dossier static	Le stockage a été isolé, les imports temporaires sont supprimés et la galerie utilise une pagination.
Le module audio dépend de ffmpeg	L'application détecte l'état de l'audio et reste utilisable même quand ffmpeg n'est pas disponible.
Les tests devaient suivre la nouvelle architecture	Les chemins de patch et les assertions ont été adaptés après la séparation des routes.

7. Équipe et répartition du travail

Photo	Membre	Rôle
	Aya EL Amrani	Structure Flask, configuration, formulaires, stockage et routes.
	Khadija Baskar	Textes français, libellés, cohérence des intitulés et contenu des pages.
	Hossam OUammi	Intégration Flask, interface, médias, galerie et pages de présentation.
	Abderrahmane El Garti	Tests fonctionnels, documentation, rapport et analyse technique.

Aya EL Amrani

Date	Commit	Message
2026-03-04	38dcb88	Configurer le studio 1
2026-03-09	e9530e0	Configurer le studio 2
2026-03-13	554b29d	Configurer le studio 3
2026-03-18	a916a2c	Configurer le studio 4
2026-03-21	36b6cd5	Structurer les formulaires 1

Khadija Baskar

Date	Commit	Message
2026-03-05	f070d4a	Renseigner les libelles 1
2026-03-10	7a89d74	Renseigner les libelles 2
2026-03-15	ed9c1e3	Renseigner les libelles 3
2026-03-20	5572214	Renseigner les libelles 4
2026-03-27	534b7c2	Renseigner les libelles 5

Hossam OUammi

Date	Commit	Message
2026-03-05	3abd34a	Monter la fabrique Flask 1
2026-03-06	318e017	Monter la fabrique Flask 2
2026-03-12	d1afc55	Monter la fabrique Flask 3
2026-03-14	e0362ff	Monter la fabrique Flask 4
2026-03-19	1713a6b	Monter la fabrique Flask 5

Abderrahmane El Garti

Date	Commit	Message
2026-03-05	4435dcb	Rediger analyse projet 1
2026-03-11	e4ff2a0	Rediger analyse projet 2
2026-03-17	81813a2	Rediger analyse projet 3
2026-03-24	e1b497b	Rediger analyse projet 4
2026-03-28	c4d2401	Rediger analyse projet 5

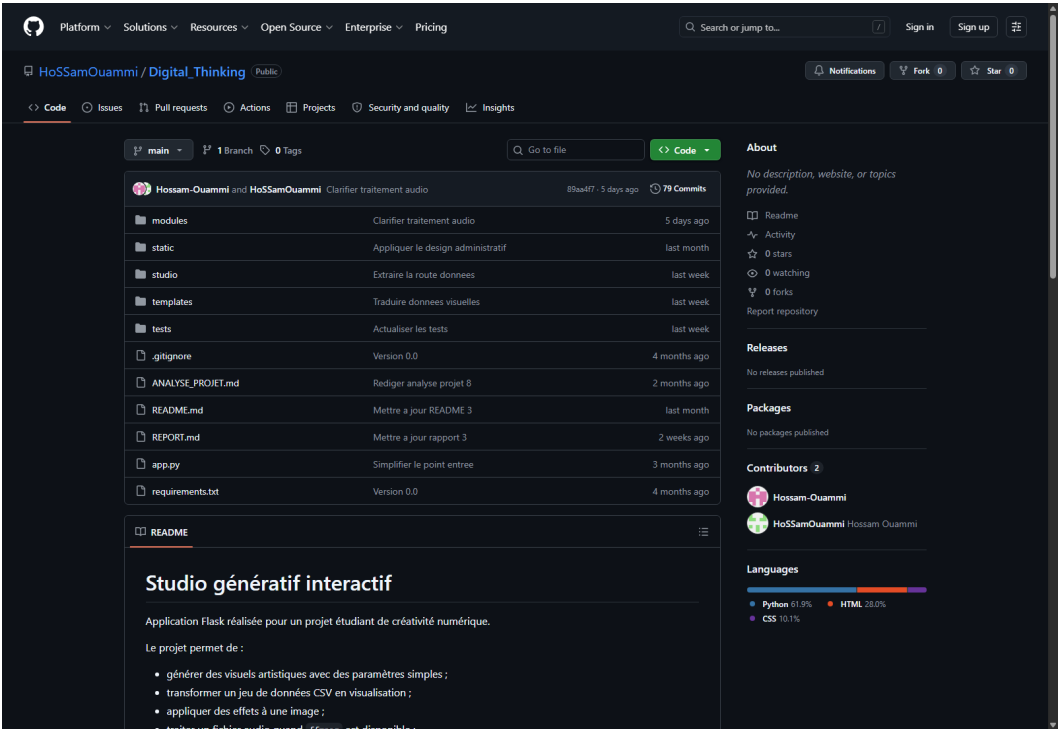
8. Validation

Point vérifié	Résultat
Tests	La suite unittest couvre les routes principales, la génération, les formulaires protégés et le nettoyage.
Application	Les pages principales se chargent correctement: accueil, équipe, génératif, données, médias et galerie.
Exports	Les fichiers générés sont sauvegardés dans static/generated puis visibles dans la galerie.
Dépôt	Le dépôt GitHub contient le code, le README, le rapport, les images d'équipe et les captures utilisées.

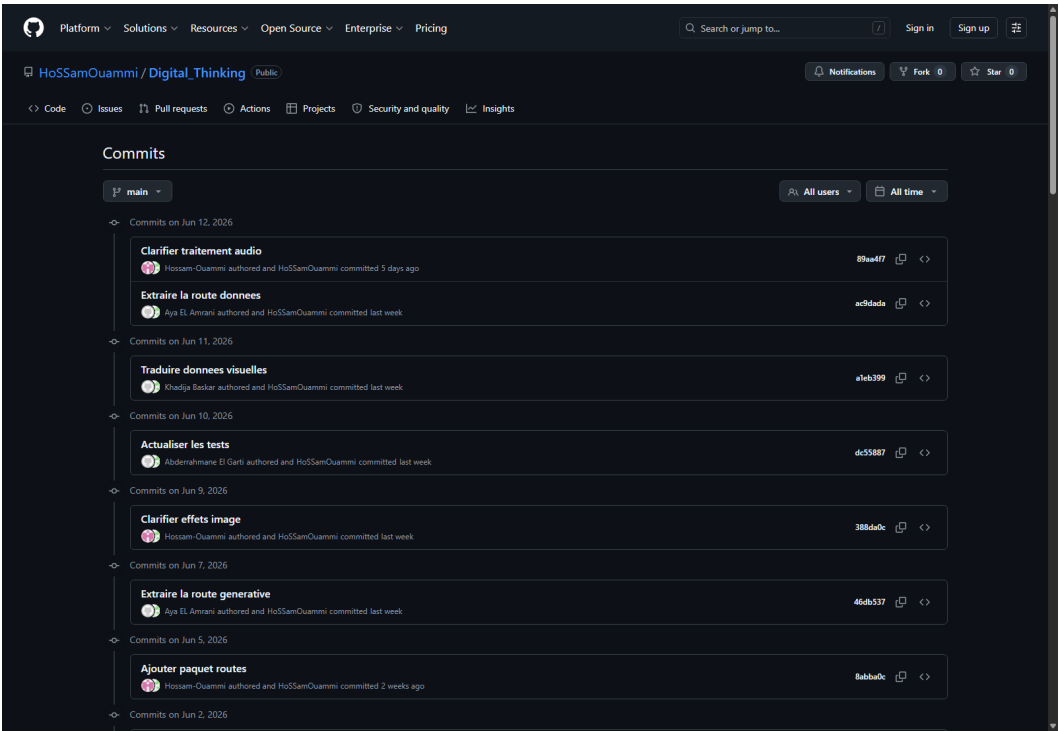
9. Suivi GitHub

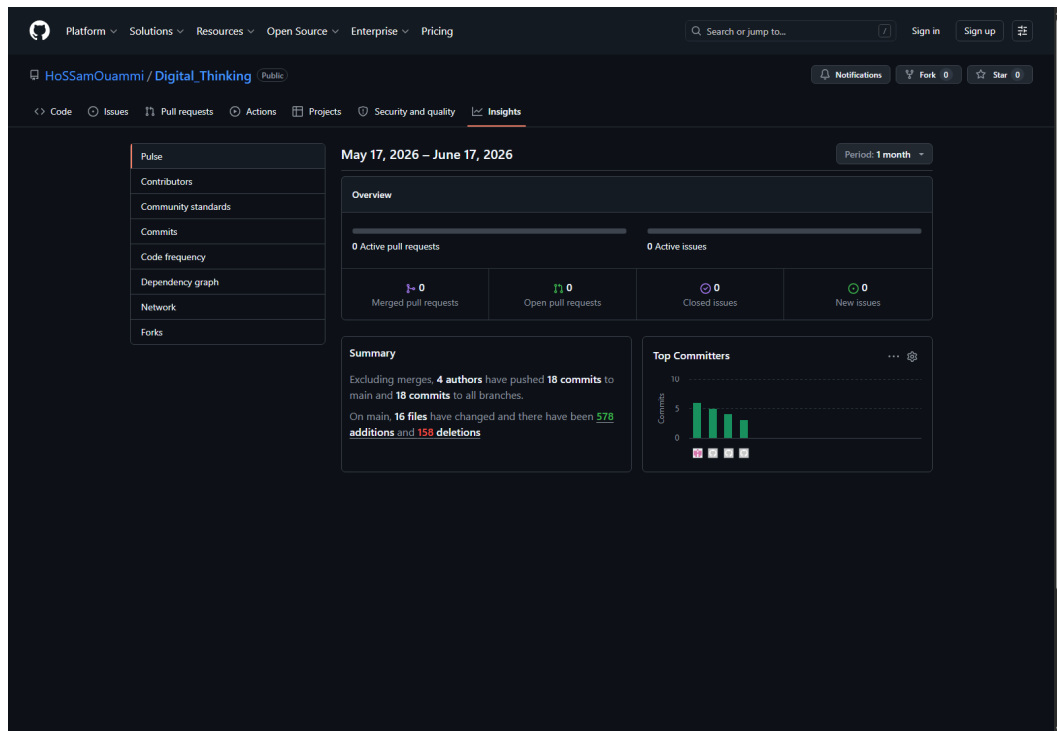
Les captures GitHub montrent le dépôt, l'historique et l'activité du projet. Elles servent surtout de preuve visuelle du suivi et de la présence des livrables dans le dépôt.

Capture GitHub - dépôt



Capture GitHub - historique des commits





10. Conclusion

La version finale reste simple, mais c'est justement son intérêt. Elle montre une application Flask complète avec plusieurs modules, une interface cohérente, des exports, une galerie, des tests et un dépôt propre. Le projet est donc présentable autant côté code que côté usage.